Object-oriented Programming

Week 6 | Lecture 1

Inheritance

- A form of software reusability where a class *inherits* an existing class' behavior and enhances it by adding more functionalities

- The existing class is called base class (or sometimes super class) and the new class is referred to as derived class (or sometimes sub class)

Example

- The *is-a* relationship represents inheritance

- The *car* is a vehicle, so any attributes and behaviors of a *vehicle* are also attributes and behaviors of a *car*

Example

```
class Vehicle
{
    // data members of base class
}

class Car: public Vehicle
{
    //data members of derived class
}
```

Base & Derived Classes

- Every derived-class object is also an object of its base class, and one base class can have many derived classes

- A derived class can access all non-private members of its base class

Visibility of Base Class Members

- A derived class can use the access modifiers public, protected or private to restrict access to its base class members

- In all situations, a derived class can never access private members of its base class

Protected

- Protected access modifier is similar to that of private access modifiers, the difference is that the class member declared as Protected are inaccessible outside the class but they can be accessed by any subclass(derived class) of that class.

Protected

```
#include <iostream>
using namespace std;
class A{
public:
        int xPublic;
protected:
        int xProtected;
private:
        int xPrivate;
};
int main()
{
        A a;
        a.xPublic = 0;
        a.xProtected = 0; // error: inaccessible
        a.xPrivate = 0;   // error: inaccessible
}
```

Access Modes in C++ Inheritance

- In C++ Inheritance, we can derive a child class from the base class in different access modes.

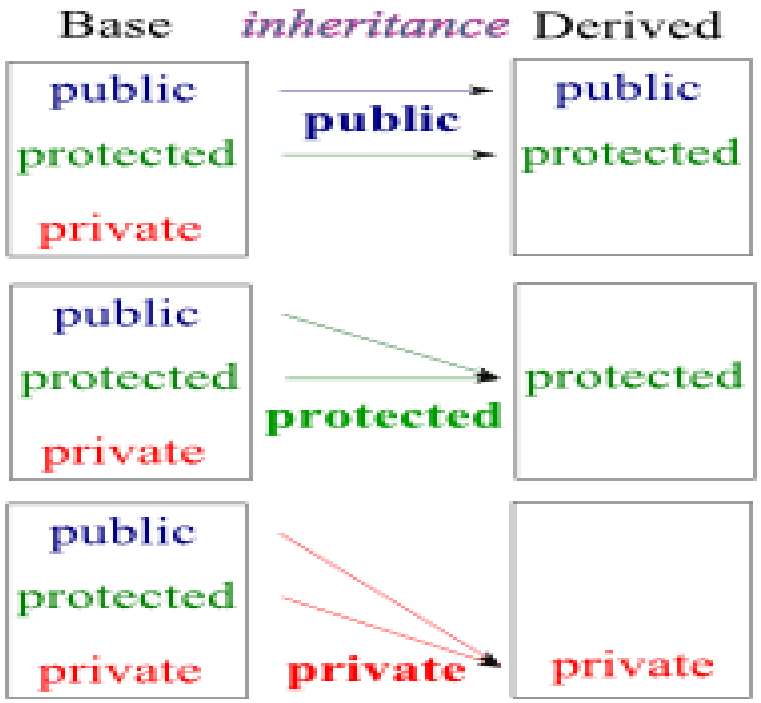
```
class Derived : Access Specifier Base {


.... ... ....



};
```

Access Modes in C++ Inheritance



Public Inheritance

- The use of access modifier public in derived class header

Example:

```
class myDerived: public myBase
{
        // derived class members
}
```

Public Inheritance

- In public inheritance:
  - The **public** members of a base class are treated as **public** members of the derived class by other classes further down the hierarchy
  - The **protected** members of a base class are treated as **protected** members of the derived class by other classes further down the hierarchy

Public Inheritance

```
class Parent
{
        private:        int a;
        public:         int b;
        protected:      int c;
}

class Child: public Parent
{
    // can never access a directly
    // can access b & c directly
}
```

```
class GrandChild: public Child
{
    // can never access a directly
    // can access b directly
    // can access c directly
}
```

Protected Inheritance

- The use of access modifier protected in derived class header

Example:

```
class myDerived: protected myBase
{
        // derived class members
}
```

Protected Inheritance

- In protected inheritance:
  - The **public** members of a base class are treated as **protected** members of the derived class by other classes further down the hierarchy
  - The **protected** members of a base class are treated as **protected** members of the derived class by other classes further down the hierarchy

Protected Inheritance

```
class Parent
{
        private:        int a;
        public:         int b;
        protected:      int c;
}

class Child: protected Parent
{
    // can never access a directly
    // can access b & c directly
}
```

```
class GrandChild: public Child
{
    // can never access a directly
    // can access b directly
    // can access c directly
}
```

Private Inheritance

- The use of access modifier private in derived class header

Example:

```
class myDerived: private myBase
{
        // derived class members
}
```

Private Inheritance

- In private inheritance:
  - All **public** & **protected** members of a base class are treated as **private** members of the derived class by other classes further down the hierarchy
  - In other words, these members can be seen as *locked* and cannot be accessed further down the hierarchy

Private Inheritance

```
class Parent
{
        private:        int a;
        public:         int b;
        protected:      int c;
}

class Child: private Parent
{
     // can never access a directly
    // can access b & c directly
}
```

```
class GrandChild: public Child
 {
   // can never access a directly
   // cannot access b directly
    // cannot access c directly
 }
```

```
class A
{
public:
        int xPublic;
protected:
        int xProtected;
private:
        int xPrivate;
};
class B : public A
{};
int main()
{
        A a;
        a.xPublic = 0;
        a.xProtected = 0; // error: inaccessible
        a.xPrivate = 0;   // error: inaccessible

        B b;
        b.xPublic = 0;
        b.xProtected = 0;  // error: inaccessible
        b.xPrivate = 0;    // error: inaccessible

        return 0;       }
```

```
#include <iostream>
using namespace std;
class A{
public:
   int xPublic;
protected:
   int xProtected;
private:
   int xPrivate;};
class B : public A{
public:
   void accessMembers()
   {
      xPublic = 10;   // Allowed: Public remains public
      xProtected = 20; // Allowed: Protected remains protected
      // xPrivate = 30; // Not Allowed: Private is inaccessible
   }};
int main(){
   B b;
   b.xPublic = 5;   // Allowed
   // b.xProtected = 10; // Not Allowed (protected members can't be accessed through objects)
   // b.xPrivate = 15;   // Not Allowed
   return 0;}
```

```
#include <iostream>
using namespace std;

class A
{
public:
   int xPublic;
protected:
   int xProtected;
private:
   int xPrivate;
};

class B : public A
{
public:
   int y = xPublic + 10;    // ? Allowed (xPublic is accessible)
   int z = xProtected + 20; // ? Allowed (xProtected is accessible)
   // int w = xPrivate + 30; // ? Not Allowed (xPrivate is inaccessible)
};
int main(){
   B b;
   cout << "y: " << b.y << endl;  // ? Accessible (xPublic is public)
   // cout << "z: " << b.z << endl; // ? Error: xProtected is inaccessible in main()
   return 0;}
```

Next Lecture

- Types of Inheritance
- Function Overloading